

Student 2 **Chintha Abhilash Reddy** 192525462RC

4 / 4 submitted

Q1. Intelligent Customer Feedback Analysis

CODE

```
import re

def preprocess(text):
    text = text.lower().strip()
    text = re.sub(r'^a-z\s', '', text)
    return text

def analyse(text):
    positive = ["good", "great", "excellent", "awesome", "happy", "love"]
    negative = ["bad", "poor", "worst", "hate", "slow", "terrible"]

    p = 0
    n = 0

    words = text.split()

    for word in words:
        if word in positive:
            p += 1
        elif word in negative:
            n += 1

    print("Positive Count:", p)
    print("Negative Count:", n)

    if p > n:
        print("Sentiment: Positive")
    elif n > p:
        print("Sentiment: Negative")
    else:
        print("Sentiment: Neutral")

num = int(input())

for i in range(num):
    feedback = input()
    feedback = preprocess(feedback)
    analyse(feedback)
```

INPUT

```
3
This product is Excellent And Awesome!
Bad quality and slow delivery.
Good product and but bad packing.
```

OUTPUT

```
Positive Count: 2
Negative Count: 0
Sentiment: Positive
Positive Count: 0
Negative Count: 2
Sentiment: Negative
Positive Count: 1
Negative Count: 1
Sentiment: Neutral
```

7/29/26, 8:35 PM

Student Submissions — Scenario Based Program Writing

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q2. Comprehensive Password Strength Checker

CODE

```
def check_password(pwd):
    if len(pwd) == 0:
        print("Weak")
        print("Password cannot be empty")
        return

    upper = lower = digit = special = False

    for ch in pwd:
        if ch.isupper():
            upper = True
        elif ch.islower():
            lower = True
        elif ch.isdigit():
            digit = True
        else:
            special = True

    score = sum([upper, lower, digit, special])

    # Strength classification
    if len(pwd) < 8 or score == 1:
        strength = "Weak"
    elif score == 4 and len(pwd) ≥ 12:
        strength = "Very Strong"
    elif score == 4:
        strength = "Strong"
    elif score ≥ 3:
        strength = "Moderate"
    else:
        strength = "Weak"

    print(strength)

    # Feedback
    if not upper:
        print("Add uppercase letter")
    if not lower:
        print("Add lowercase letter")
    if not digit:
        print("Add digit")
    if not special:
        print("Add special character")

password = input().strip()
check_password(password)
```

INPUT

Password@123

OUTPUT

Very Strong

Q3. Structured Server Log Error Analyser

CODE

```
def extract_errors(logs):
    errors = []
    for line in logs:
        t = line.strip().lower()
        if "error" in t:
            errors.append(line.strip())
    return errors

def remove_duplicates(errs):
    u = []
    for e in errs:
        if e not in u:
            u.append(e)
    return u

def main():
    n = int(input())
    logs = [input() for _ in range(n)]
    errors = extract_errors(logs)
    unique = remove_duplicates(errors)

    counts = {}
    for e in errors:
        counts[e] = counts.get(e, 0) + 1

    print("Error Summary")
    print("-----")
    print("Total Error Entries:", len(errors))
    print("Unique Errors:", len(unique))
    for e in unique:
        print(e)
        print("Count:", counts[e])
        print()

main()
```

INPUT

```
7
info Server Started
Error File Not Found
Warning Low Memory
Error Database Failed
Info Login Success
Error File Not Found
Error Disk Full
```

OUTPUT

```
Error Summary
-----
Total Error Entries: 4
Unique Errors: 3
Error File Not Found
Count: 2

Error Database Failed
Count: 1

Error Disk Full
Count: 1
```

7/29/26, 8:35 PM

Student Submissions — Scenario Based Program Writing

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q4. Email Domain Frequency Analyser

CODE

```
def get_domain(email):
    email = email.strip().lower()
    if email.count('@') != 1:
        return None
    user, domain = email.split('@')
    if not user or not domain or '.' not in domain:
        return None
    if domain.startswith('.') or domain.endswith('.'):
        return None
    return domain

freq = {}

while True:
    try:
        s = input()
    except EOFError:
        break

    d = get_domain(s)
    if d:
        freq[d] = freq.get(d, 0) + 1

if not freq:
    print("No valid email addresses")
else:
    for d, c in freq.items():
        print(d, c)

    max_domain = max(freq, key=freq.get)
    print(max_domain)
    print(freq[max_domain])
```

INPUT

```
abc@gmail.com
xyz@gmail.com
ram@gmail.com
invalidemail
```

OUTPUT

```
gmail.com 3
gmail.com
3
```